TypeScript

The Complete 80/20 Guide

Basic → Advanced + 50 Real-World Solutions

Everything the industry actually uses

Introduction & The 80/20 Principle

TypeScript is JavaScript with static types. 80% of your day-to-day TypeScript work relies on 20% of its features. This guide teaches that critical 20% deeply, then walks you through the advanced remaining features — and arms you with 50 battle-tested solutions to the most common TypeScript errors developers encounter.

Category	What You'll Learn
Fundamentals	Types, interfaces, type aliases, functions, enums
Intermediate	Generics, utility types, type narrowing, modules
Advanced	Conditional types, mapped types, decorators, infer
Ecosystem	tsconfig, strict mode, ESLint, React+TS, Node+TS
50 Solutions	The most common TypeScript errors & how to fix them

PART 1 — FUNDAMENTALS

1. Basic Types — The Daily Drivers

The types you will use in literally every TypeScript file:

```
// Primitive types
let name: string = 'Alice';
let age: number = 30;
let active: boolean = true;
let nothing: null = null;
let missing: undefined = undefined;

// Arrays
let scores: number[] = [1, 2, 3];
let tags: Array<string> = ['ts', 'js'];

// Tuple — fixed-length array with known types
let coord: [number, number] = [40.7, -74.0];

// any — escape hatch (use sparingly!)
let wild: any = 'could be anything';

// unknown — safer alternative to any
let input: unknown = getUserInput();
if (typeof input === 'string') console.log(input.toUpperCase()); // narrowed

// void — functions that return nothing
function log(msg: string): void { console.log(msg); }

// never — functions that never return
function fail(msg: string): never { throw new Error(msg); }
```

> **ℹ INFO**
> The 80/20 rule: string, number, boolean, arrays, and unknown cover ~80% of all type annotations you will ever write.

2. Interfaces vs Type Aliases

Both define the shape of an object. Prefer interfaces for public APIs and extendable structures; prefer type aliases for unions, intersections, and utility types.

```
// Interface — extendable, mergeable
interface User {
  id: number;
  name: string;
  email?: string;  // optional
  readonly createdAt: Date; // read-only
}

// Extending
interface Admin extends User {
  role: 'admin' | 'superadmin';
}

// Type Alias — more flexible
type ID = string | number;
type Point = { x: number; y: number };
```

```
// Intersection type
type AdminUser = User & { permissions: string[] };

// Union type
type Status = 'active' | 'inactive' | 'banned';
```

Feature	Interface	Type Alias
Extend / Inherit	extends keyword	& intersection
Declaration Merging	Yes (same name = merge)	No (error)
Union / Intersection	No	Yes
Computed Properties	No	Yes
Best For	Object shapes, OOP	Unions, utilities, functions

3. Functions — Types & Signatures

```
// Basic typed function
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function
const multiply = (a: number, b: number): number => a * b;

// Optional & default parameters
function greet(name: string, title?: string, prefix = 'Hello'): string {
  return `${prefix}, ${title ?? ''} ${name}`;
}

// Rest parameters
function sum(...nums: number[]): number {
  return nums.reduce((a, b) => a + b, 0);
}

// Function type
type Transformer = (value: string) => string;
const upper: Transformer = (v) => v.toUpperCase();

// Overloads
function format(value: string): string;
function format(value: number, decimals: number): string;
function format(value: any, decimals = 2): string {
  if (typeof value === 'string') return value.trim();
  return value.toFixed(decimals);
}
```

4. Classes

```
class Animal {
  // Access modifiers: public (default), private, protected
  constructor(
    public name: string,
    private age: number,
    protected species: string
  ) {}
```

```
  // Getter
  get info(): string {
    return `${this.name} (${this.species})`;
  }

  // Static method
  static create(name: string): Animal {
    return new Animal(name, 0, 'Unknown');
  }

  speak(): void {
    console.log(`${this.name} makes a noise.`);
  }
}

class Dog extends Animal {
  constructor(name: string, private breed: string) {
    super(name, 0, 'Canis lupus');
  }
  speak(): void { console.log('Woof!'); }
}

// Implementing an interface
interface Serializable { serialize(): string; }
class Config implements Serializable {
  serialize(): string { return JSON.stringify(this); }
}
```

5. Enums

```
// Numeric enum (default)
enum Direction { Up, Down, Left, Right }  // 0,1,2,3

// String enum (preferred — more debuggable)
enum Status {
  Active  = 'ACTIVE',
  Inactive = 'INACTIVE',
  Banned  = 'BANNED',
}

// Const enum — erased at compile time, zero overhead
const enum HttpMethod {
  GET = 'GET',
  POST = 'POST',
  PUT = 'PUT',
  DELETE = 'DELETE',
}

function sendRequest(url: string, method: HttpMethod) { /* ... */ }
sendRequest('/api/users', HttpMethod.GET);
```

⚠ **TIP**

Prefer string enums over numeric for readability in logs/debugging. Use const enum for performance-critical code.

PART 2 — INTERMEDIATE

6. Generics — Write Once, Type Many

Generics let you write reusable components that work with any type while preserving type safety.

```
// Generic function
function identity<T>(arg: T): T { return arg; }
identity<string>('hello');  // explicit
identity(42);               // inferred: T = number

// Generic interface
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

// Usage
const res: ApiResponse<User[]> = await fetchUsers();

// Generic with constraint
function getLength<T extends { length: number }>(arg: T): number {
  return arg.length;
}
getLength('hello');      // 5
getLength([1,2,3]);      // 3

// Multiple type parameters
function pair<K, V>(key: K, value: V): [K, V] {
  return [key, value];
}

// Generic class
class Stack<T> {
  private items: T[] = [];
  push(item: T): void { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items[this.items.length - 1]; }
}
const numStack = new Stack<number>();
```

7. Utility Types — The 80/20 Swiss Army Knife

These built-in types handle ~80% of type transformation needs:

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// Partial — all properties optional
type UserUpdate = Partial<User>;
// { id?: number; name?: string; email?: string; age?: number }

// Required — all properties required
type UserRequired = Required<UserUpdate>;
```

```
// Readonly — all properties immutable
type ImmutableUser = Readonly<User>;

// Pick — select specific properties
type UserPreview = Pick<User, 'id' | 'name'>;

// Omit — exclude specific properties
type UserWithoutId = Omit<User, 'id'>;

// Record — map keys to values
type RoleMap = Record<string, User[]>;

// Exclude — remove from union
type NoString = Exclude<string | number | boolean, string>;  // number | boolean

// Extract — keep from union
type OnlyString = Extract<string | number | boolean, string>;  // string

// NonNullable — remove null and undefined
type Clean = NonNullable<string | null | undefined>;  // string

// ReturnType — get a function's return type
function fetchUser(): Promise<User> { /* ... */ return Promise.resolve({} as User); }
type FetchResult = ReturnType<typeof fetchUser>;  // Promise<User>

// Parameters — get function params as tuple
type FetchParams = Parameters<typeof fetch>;  // [input: RequestInfo, init?: RequestInit]

// Awaited — unwrap Promise
type ResolvedUser = Awaited<Promise<User>>;  // User
```

Utility Type	What It Does	Common Use Case
Partial<T>	All props optional	Update/PATCH payloads
Required<T>	All props required	Validated forms
Readonly<T>	All props immutable	Redux state, config
Pick<T,K>	Select props	View models, DTOs
Omit<T,K>	Exclude props	Remove sensitive fields
Record<K,V>	Object type from keys	Lookup maps, dictionaries
Exclude<T,U>	Remove from union	Filter union members
NonNullable<T>	Remove null/undefined	After null checks
ReturnType<T>	Infer return type	Auto-infer function results
Awaited<T>	Unwrap Promise	Async return types

8. Type Narrowing

TypeScript narrows union types based on runtime checks — this is called control flow analysis.

```
type Shape = Circle | Square;
interface Circle { kind: 'circle'; radius: number; }
interface Square { kind: 'square'; side: number; }

// 1. typeof guard
function process(value: string | number) {
  if (typeof value === 'string') {
```

```
    return value.toUpperCase(); // string here
  }
  return value.toFixed(2); // number here
}

// 2. instanceof guard
function handleError(error: Error | string) {
  if (error instanceof Error) {
    console.error(error.message); // Error here
  } else {
    console.error(error); // string here
  }
}

// 3. Discriminated union (most powerful)
function area(shape: Shape): number {
  switch (shape.kind) {
    case 'circle': return Math.PI * shape.radius ** 2;
    case 'square': return shape.side ** 2;
  }
}

// 4. Type guard function
function isUser(obj: any): obj is User {
  return typeof obj === 'object' && 'id' in obj && 'name' in obj;
}

// 5. Assertion function
function assertDefined<T>(val: T | null | undefined): asserts val is T {
  if (val == null) throw new Error('Expected defined value');
}
```

9. Modules & Declaration Files

```
// Exporting
export interface Product { id: string; name: string; price: number; }
export type Currency = 'USD' | 'EUR' | 'GBP';
export const TAX_RATE = 0.08;
export default class Cart { /* ... */ }

// Importing
import Cart, { Product, TAX_RATE } from './cart';
import type { Currency } from './cart'; // type-only import

// Re-exporting
export { Product } from './product';
export * from './utils';

// Declaration file (.d.ts) — typing untyped JS libs
declare module 'my-legacy-lib' {
  export function doThing(input: string): number;
  export const version: string;
}

// Ambient declarations
declare const __DEV__: boolean;
declare function require(module: string): any;
```

PART 3 — ADVANCED

10. Conditional Types

```
// Basic conditional type: T extends U ? X : Y
type IsString<T> = T extends string ? true : false;
type A = IsString<string>;  // true
type B = IsString<number>;  // false

// infer — extract type from another
type UnpackPromise<T> = T extends Promise<infer U> ? U : T;
type Resolved = UnpackPromise<Promise<User>>;  // User

type UnpackArray<T> = T extends (infer U)[] ? U : T;
type Element = UnpackArray<string[]>;  // string

// Distributive conditional types
type ToArray<T> = T extends any ? T[] : never;
type StringOrNumberArr = ToArray<string | number>;
// string[] | number[]  (distributes over union)

// Practical: deep readonly
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
};
```

11. Mapped Types

```
// Create new type by mapping over keys
type Optional<T> = { [K in keyof T]?: T[K] };
type Mutable<T>  = { -readonly [K in keyof T]: T[K] };  // remove readonly
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Remap keys with 'as'
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
type UserGetters = Getters<User>;
// { getId: () => number; getName: () => string; ... }

// Filter keys by type
type StringKeys<T> = {
  [K in keyof T]: T[K] extends string ? K : never;
}[keyof T];
type UserStringFields = StringKeys<User>;  // 'name' | 'email'
```

12. Template Literal Types

```
type Direction = 'top' | 'right' | 'bottom' | 'left';
type CSSPadding = `padding-${Direction}`;
// 'padding-top' | 'padding-right' | 'padding-bottom' | 'padding-left'

type EventName<T extends string> = `on${Capitalize<T>}`;
type ClickEvent = EventName<'click'>;  // 'onClick'

// Route builder
type Route = `/api/${string}`;
```

```
const valid: Route = '/api/users';      // OK
// const invalid: Route = '/users';     // Error!

// Utility string types
type Upper = Uppercase<'hello'>;    // 'HELLO'
type Lower = Lowercase<'HELLO'>;    // 'hello'
type Cap   = Capitalize<'hello'>;   // 'Hello'
type Uncap = Uncapitalize<'Hello'>; // 'hello'
```

13. Decorators (Stage 3)

Decorators are heavily used in frameworks like NestJS, Angular, TypeORM, and class-validator.

```
// tsconfig: { "experimentalDecorators": true }

// Class decorator
function Injectable(target: Function) {
  // registers class for dependency injection
}

// Method decorator
function Log(target: any, key: string, descriptor: PropertyDescriptor) {
  const original = descriptor.value;
  descriptor.value = function(...args: any[]) {
    console.log(`Calling ${key} with`, args);
    return original.apply(this, args);
  };
  return descriptor;
}

// Property decorator
function Required(target: any, key: string) {
  // validation logic
}

// NestJS-style example
@Injectable()
class UserService {
  @Log
  async findUser(id: number): Promise<User> {
    return {} as User;
  }

  @Required
  name!: string;
}
```

14. Advanced Patterns

```
// Builder pattern with fluent API
class QueryBuilder<T> {
  private filters: Partial<T> = {};
  where<K extends keyof T>(key: K, value: T[K]): this {
    this.filters[key] = value;
    return this;
  }
  build(): Partial<T> { return this.filters; }
}

// Branded types — prevent mixing compatible primitives
type UserId    = string & { readonly __brand: 'UserId' };
```

```
type ProductId = string & { readonly __brand: 'ProductId' };
function makeUserId(id: string): UserId { return id as UserId; }
const uid = makeUserId('u-123');
// Can't accidentally pass ProductId where UserId is expected

// Opaque types for strict domain modeling
type Email = string & { _email: void };
function validateEmail(s: string): Email {
  if (!s.includes('@')) throw new Error('Invalid email');
  return s as Email;
}

// Satisfies operator (TS 4.9+)
const palette = {
  red:   [255, 0, 0],
  green: '#00ff00',
} satisfies Record<string, string | number[]>;
// palette.red is still number[] (not string | number[])
```

PART 4 — ECOSYSTEM & CONFIGURATION

15. tsconfig.json — The Most Important Options

```
{
  "compilerOptions": {
    // Target & module
    "target": "ES2022",          // output JS version
    "module": "NodeNext",        // module system
    "moduleResolution": "NodeNext",
    "lib": ["ES2022", "DOM"],    // available globals

    // Strictness (always enable in new projects)
    "strict": true,              // enables all strict checks
    "noUncheckedIndexedAccess": true, // arr[i] is T | undefined
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,

    // Output
    "outDir": "./dist",
    "rootDir": "./src",
    "declaration": true,         // generate .d.ts files
    "declarationMap": true,
    "sourceMap": true,

    // Imports
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,

    // Misc
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

16. TypeScript with React

```
// Component props
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
  disabled?: boolean;
  children?: React.ReactNode;
}

const Button: React.FC<ButtonProps> = ({
  label, onClick, variant = 'primary', disabled, children
}) => <button onClick={onClick} disabled={disabled}>{children ?? label}</button>;

// useState
const [user, setUser] = React.useState<User | null>(null);
```

```
// useRef
const inputRef = React.useRef<HTMLInputElement>(null);

// useReducer
type Action = { type: 'INCREMENT' } | { type: 'SET'; payload: number };
function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'INCREMENT': return state + 1;
    case 'SET':       return action.payload;
  }
}

// Custom hook
function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = React.useState<T>(() => {
    try { return JSON.parse(localStorage.getItem(key) ?? '') ?? initial; }
    catch { return initial; }
  });
  const set = (v: T) => { setValue(v); localStorage.setItem(key, JSON.stringify(v)); };
  return [value, set] as const;
}

// Generic component
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}
function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return <ul>{items.map(item => <li
key={keyExtractor(item)}>{renderItem(item)}</li>)}</ul>;
}
```

17. TypeScript with Node.js / Express

```
// npm i -D @types/node @types/express
import express, { Request, Response, NextFunction } from 'express';

// Typed request body
interface CreateUserBody { name: string; email: string; }

app.post('/users', async (
  req: Request<{}, {}, CreateUserBody>,
  res: Response<User>,
  next: NextFunction
) => {
  const { name, email } = req.body; // fully typed
  const user = await createUser({ name, email });
  res.status(201).json(user);
});

// Typed env variables
interface Env {
  PORT: string;
  DATABASE_URL: string;
  JWT_SECRET: string;
}
const env = process.env as unknown as Env;

// Or use zod for runtime validation
import { z } from 'zod';
const EnvSchema = z.object({
```

```
  PORT: z.string().default('3000'),
  DATABASE_URL: z.string(),
});
const env2 = EnvSchema.parse(process.env);
```

PART 5 — 50 SOLUTIONS TO COMMON TYPESCRIPT PROBLEMS

#01 — Property does not exist on type

Solution
Use optional chaining, type assertion, or add the property to the interface.

```
interface User { name: string; }
const u = {} as User;
// u.age // Error: Property 'age' does not exist
// Fix: extend interface or use optional
const age = (u as any).age;          // escape hatch
const age2 = 'age' in u ? u.age : undefined; // type-safe
```

#02 — Object is possibly null or undefined

Solution
Use optional chaining (?.), nullish coalescing (??), or type guards to handle nulls.

```
// Bad
const len = user.name.length; // Error if user is null
// Good
const len = user?.name?.length ?? 0;
// Or narrow with guard
if (user != null) { const len = user.name.length; }
```

#03 — Type 'X' is not assignable to type 'Y'

Solution
Ensure types match or use type assertion as a last resort. Often caused by missing properties.

```
interface Cat { name: string; lives: number; }
const obj = { name: 'Whiskers' };
// const c: Cat = obj; // Error: missing 'lives'
const c: Cat = { ...obj, lives: 9 }; // Fix: add missing prop
```

#04 — Argument of type 'string | undefined' is not assignable

Solution
Provide a fallback or narrow the type before passing it to a function.

```
function greet(name: string) { console.log('Hello ' + name); }
const name: string | undefined = getName();
// greet(name); // Error
greet(name ?? 'stranger');  // Fix: fallback
if (name) greet(name);      // Fix: guard
```

#05 — Cannot find name 'X'

Solution
Add the missing lib to tsconfig, install @types package, or add declare statement.

```
// In tsconfig.json
{ "compilerOptions": { "lib": ["ES2022","DOM"] } }
// For node globals
npm install -D @types/node
// Quick declare
declare const __VERSION__: string;
```

#06 — 'this' implicitly has type 'any'

Solution
Add an explicit type for 'this' as the first parameter, or use arrow functions.

```
// Bad
function getName() { return this.name; } // Error
// Fix: type 'this' parameter
function getName(this: User): string { return this.name; }
// Or use arrow function in class
class Foo { name='foo'; get = () => this.name; }
```

#07 — No overload matches this call

Solution
Check you are passing the right combination of arguments. TypeScript overloads are checked top-to-bottom.

```
function fmt(val: string): string;
function fmt(val: number, dec: number): string;
function fmt(val: any, dec = 2): string { return String(val); }
fmt('hello');     // OK
fmt(3.14, 2);    // OK
// fmt(3.14);    // Error — no overload for (number) alone
```

#08 — Element implicitly has an 'any' type (index signature)

Solution
Add an index signature to the type, use Record<>, or use a type assertion.

```
const map: Record<string, number> = { a:1 };
const val = map['a']; // number
// Object without index sig:
const obj = { name: 'Alice' };
const key: string = 'name';
const v = obj[key as keyof typeof obj]; // Fix
```

#09 — Type 'never' error / exhaustive check failure

Solution
Add a case for every union member, or use an exhaustive check function.

```
type Shape = 'circle' | 'square' | 'triangle';
function area(s: Shape): number {
  if (s === 'circle') return 1;
  if (s === 'square') return 2;
  // TypeScript knows s is 'triangle' here
  const _exhaustive: never = s; // compile error if missed
  throw new Error('Unhandled shape');
}
```

#10 — Strict null checks breaking existing code

Solution
Enable strictNullChecks incrementally. Use non-null assertion (!) only when you are certain.

```
// Non-null assertion (use sparingly)
const el = document.getElementById('app')!;
// Better: check first
const el2 = document.getElementById('app');
if (!el2) throw new Error('Element not found');
el2.innerHTML = 'Hello'; // el2 is HTMLElement here
```

#11 — Generic type parameter not inferred correctly

Solution
Provide explicit type arguments when inference fails, or add constraints.

```
function wrap<T>(val: T) { return { value: val }; }
const r = wrap<string>('hello'); // explicit
const r2 = wrap('hello');        // inferred OK
// When inference breaks:
function parse<T = unknown>(json: string): T {
  return JSON.parse(json);
}
const user = parse<User>(jsonStr); // must be explicit
```

#12 — Cannot use type parameter as constraint on itself

Solution
Use a separate constraint or split into two parameters.

```
// Bad: T extends T[keyof T] — circular
// Good:
function pick<T, K extends keyof T>(obj: T, keys: K[]): Pick<T,K> {
  return keys.reduce((acc, k) => ({ ...acc, [k]: obj[k] }), {} as Pick<T,K>);
}
```

#13 — Promise return type mismatch

Solution
Ensure all code paths return a value, or explicitly annotate Promise<T>.

```
async function fetchUser(id: number): Promise<User | null> {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) return null;
    return res.json() as Promise<User>;
  } catch { return null; }
}
```

#14 — Unhandled promise rejection at type level

Solution
Always type the return of async functions. Use try/catch or .catch().

```
// Always annotate async functions
async function save(data: User): Promise<void> {
  await db.save(data); // TypeScript won't warn on unhandled rejection
}
// Use Result pattern for errors
type Result<T,E=Error> = { ok:true; value:T } | { ok:false; error:E };
```

#15 — await has no effect on this expression

Solution
You are awaiting a non-Promise value. Remove await or fix the function to be async.

```
// Bad: value is not a Promise
const name = await user.name; // Error
// Fix:
const name = user.name;
// Or if it should be async:
const name = await user.getName();
```

#16 — Cannot assign to read-only property

Solution
Create a new object with the updated property instead of mutating.

```
const user: Readonly<User> = { id:1, name:'Alice' };
// user.name = 'Bob'; // Error
const updated = { ...user, name: 'Bob' }; // Fix
```

#17 — Object literal may only specify known properties

Solution

TypeScript's excess property checking rejects extra properties on literals. Assign to a variable first.

```
interface Point { x: number; y: number; }
// const p: Point = { x:1, y:2, z:3 }; // Error
const raw = { x:1, y:2, z:3 };
const p: Point = raw; // OK — excess check skipped
```

#18 — Tuple type out of bounds

Solution

Access only valid indices, or convert to a regular array if length is dynamic.

```
type Pair = [string, number];
const p: Pair = ['Alice', 30];
// p[2]; // Error: Tuple type 'Pair' of length '2' has no element at index '2'
const safe = p[0]; // string
const num  = p[1]; // number
```

#19 — noUncheckedIndexedAccess — arr[i] is T | undefined

Solution

Check for undefined before using indexed values when noUncheckedIndexedAccess is on.

```
// With noUncheckedIndexedAccess: true
const arr = [1, 2, 3];
const first = arr[0]; // number | undefined
if (first !== undefined) { console.log(first * 2); }
// Or use at() with nullish coalescing:
const val = arr.at(0) ?? 0;
```

#20 — Spreading types with incompatible signatures

Solution

Use type intersections carefully; spreading only copies enumerable own properties.

```
interface A { a: string; }
interface B { b: number; }
type AB = A & B;
const merged: AB = { ...objA, ...objB }; // OK if both shapes match
// For complex merges use Object.assign with typed result:
```

#21 — Property has no initializer and not assigned in constructor

Solution

Initialize in constructor, use definite assignment assertion (!), or set strictPropertyInitialization: false.

```
class Service {
  // Option 1: initialize
  private db: Database = new Database();
  // Option 2: definite assignment (you guarantee it will be set)
  private logger!: Logger;
  // Option 3: optional
  private cache?: Cache;
}
```

#22 — Accessing private/protected member from outside class

Solution

Use public getter methods, or reconsider your visibility modifiers.

```
class Counter {
  private count = 0;
  increment() { this.count++; }
  getCount(): number { return this.count; } // public getter
}
```

#23 — Abstract class cannot be instantiated

Solution

You must extend the abstract class and implement all abstract members.

```
abstract class Shape {
  abstract area(): number;
  describe() { return `Area is ${this.area()}`; }
}
// new Shape(); // Error
class Circle extends Shape {
  constructor(private r: number) { super(); }
  area() { return Math.PI * this.r ** 2; }
}
```

#24 — Module not found or no declaration file

Solution

Install the @types package, create a .d.ts file, or use declare module.

```
// Install types
npm install -D @types/lodash
// If no @types exists, create src/types/module-name.d.ts:
declare module 'some-untyped-lib' {
  export function doSomething(x: string): void;
}
```

#25 — import type vs import — when to use each

Solution
Use 'import type' for type-only imports to avoid runtime overhead and circular dependency issues.

```
// Type-only import — erased at compile time
import type { User } from './user';
// Regular import — value is available at runtime
import { createUser } from './user';
// Mixed (TS 4.5+)
import { createUser, type User } from './user';
```

#26 — Circular import causing type errors

Solution
Extract shared types to a separate file that both modules can import without creating a cycle.

```
// types.ts — no imports from other project files
export interface User { id: number; name: string; }
// user.service.ts
import type { User } from './types';
// auth.service.ts
import type { User } from './types'; // both import types, no cycle
```

#27 — Implicit any on function parameters

Solution
Annotate all function parameters explicitly, or enable noImplicitAny in tsconfig.

```
// Bad
function process(data) { return data.length; } // Error: implicit any
// Good
function process(data: string[]): number { return data.length; }
```

#28 — Strict function types — callback variance errors

Solution
Make callback parameters as broad as needed. Covariant return types, contravariant parameter types.

```
type Handler = (event: MouseEvent) => void;
// A function accepting Event is a valid Handler
const h: Handler = (e: Event) => {}; // OK — contravariance
// A function accepting UIEvent is also OK
const h2: Handler = (e: UIEvent) => {};
```

#29 — Typing API responses from fetch

Solution
Never trust external data. Use Zod or io-ts to parse and validate at runtime.

```
import { z } from 'zod';
const UserSchema = z.object({
  id: z.number(), name: z.string(), email: z.string().email(),
});
type User = z.infer<typeof UserSchema>; // derive type from schema
const res = await fetch('/api/user');
const user = UserSchema.parse(await res.json()); // throws if invalid
// Or safe parse:
const result = UserSchema.safeParse(data);
if (result.success) { console.log(result.data); }
```

#30 — JSON.parse returns any — how to type it safely

Solution
Use a schema validator (Zod recommended) or write a generic typed parse helper.

```
import { z } from 'zod';
function parseJSON<T>(schema: z.ZodType<T>, json: string): T {
  return schema.parse(JSON.parse(json));
}
const user = parseJSON(UserSchema, rawJson); // fully typed User
```

#31 — JSX element type does not have any construct signatures

Solution
Return JSX.Element or React.ReactElement, not component class instances.

```
// Bad
function Bad(): React.Component { return new MyComp(); }
// Good
function Good(): JSX.Element { return <MyComp />; }
// Best (TS 5+ / React 18)
function Best(): React.ReactNode { return <MyComp />; }
```

#32 — React event handler type

Solution
Use the specific React event types for all event handlers.

```
// Common React event types
onChange: React.ChangeEvent<HTMLInputElement>
onClick: React.MouseEvent<HTMLButtonElement>
onSubmit: React.FormEvent<HTMLFormElement>
onKeyDown: React.KeyboardEvent<HTMLInputElement>
// Example handler:
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setValue(e.target.value);
};
```

#33 — Generic component props with TypeScript

Solution

Use a generic function component signature instead of React.FC for generic components.

```
// Bad: React.FC doesn't support generics cleanly
// Good:
function Select<T extends string | number>({
  options, value, onChange
}: { options: T[]; value: T; onChange: (v:T) => void }) {
  return <select value={String(value)} onChange={e => onChange(e.target.value as T)}>
    {options.map(o => <option key={String(o)}>{o}</option>)}
  </select>;
}
```

#34 — useRef typed incorrectly — current is possibly null

Solution

Initialize useRef with null and assert non-null when you know it is mounted.

```
const ref = React.useRef<HTMLInputElement>(null);
// Inside useEffect (always mounted):
React.useEffect(() => {
  ref.current?.focus(); // safe: optional chaining
  ref.current!.focus(); // asserting non-null (OK post-mount)
}, []);
```

#35 — Using 'as' incorrectly causing runtime errors

Solution

Type assertions bypass checks. Validate at runtime with typeof, instanceof, or Zod.

```
// Dangerous — no runtime guarantee
const user = data as User;
// Safe — validate first
function isUser(d: unknown): d is User {
  return typeof d === 'object' && d !== null && 'id' in d;
}
if (isUser(data)) { console.log(data.id); } // safe
```

#36 — Double assertion (X as unknown as Y)

Solution

Use double assertion only as last resort for incompatible type escapes, never in production code without a comment.

```
// Last resort — document why
const el = document.querySelector('.modal') as unknown as MyModalElement;
// Better: use a type guard
function isModal(el: Element): el is MyModalElement {
  return el.classList.contains('modal');
}
```

#37 — Using keyof and typeof together

Solution

typeof gets the TypeScript type of a value; keyof gets the keys of a type.

```
const config = { host: 'localhost', port: 3000, debug: true };
type Config = typeof config;            // { host:string; port:number; debug:boolean }
type ConfigKey = keyof typeof config;   // 'host' | 'port' | 'debug'

function get<K extends keyof typeof config>(key: K): typeof config[K] {
  return config[key];
```

```
}
const port = get('port'); // number
```

#38 — Switch not exhaustive over a union

Solution
Add a default case that assigns the value to never — TypeScript will error if any union member is unhandled.

```
type Color = 'red' | 'green' | 'blue';
function toHex(c: Color): string {
  switch(c) {
    case 'red':   return '#ff0000';
    case 'green': return '#00ff00';
    case 'blue':  return '#0000ff';
    default: {
      const _never: never = c; // Error if new color added without handling
      return '#000000';
    }
  }
}
```

#39 — Numeric enum reverse mapping confusion

Solution
String enums are safer than numeric — no reverse mapping, more readable in logs.

```
enum N { A, B, C }  // N[0] === 'A', N['A'] === 0 (reverse mapping)
N[0]; // 'A' — confusing!
// Use string enum instead:
enum S { A = 'A', B = 'B', C = 'C' }
// No reverse mapping, safe to iterate with Object.values(S)
```

#40 — Augmenting third-party types (module augmentation)

Solution
Use declare module to extend existing types from third-party libraries.

```
// Add custom properties to Express Request
declare module 'express-serve-static-core' {
  interface Request {
    user?: User;
    requestId: string;
  }
}
// Now req.user is typed everywhere
```

#41 — TypeScript compilation is slow on large projects

Solution
Use project references, incremental compilation, and skipLibCheck.

```
// tsconfig.json
{ "compilerOptions": {
    "incremental": true,
    "tsBuildInfoFile": ".tsbuildinfo",
    "skipLibCheck": true
  }
}
// For monorepos: use project references
// tsconfig.json at root: { "references": [{"path":"./packages/core"}] }
```

#42 — Variance issues with generic interfaces

Solution
TypeScript uses structural typing. Make interfaces covariant in output, contravariant in input.

```
// Covariant (safe for reads)
interface Producer<T> { produce(): T; }
// Contravariant (safe for writes)
interface Consumer<T> { consume(val: T): void; }
// Invariant (both)
interface Container<T> { get(): T; set(val: T): void; }
```

#43 — Discriminated union with optional discriminant

Solution
Never make the discriminant optional — it must always be present and literal.

```
// Bad — optional discriminant
interface A { kind?: 'a'; x: number; }
// Good — always present
type Event =
  | { kind: 'click';    x: number; y: number }
  | { kind: 'keydown';  key: string }
  | { kind: 'scroll';   delta: number };
```

#44 — Organizing types across a large codebase

Solution
Co-locate types with their consumers. Use barrel files (index.ts) for public APIs only.

```
// Feature-based structure:
src/
  users/
    users.types.ts   // interfaces, enums for this feature
    users.service.ts
    users.routes.ts
    index.ts         // export only what others need
  shared/
    types.ts         // truly shared types (IDs, pagination, etc.)
```

#45 — Keeping Zod schema and TypeScript type in sync

Solution
Always derive the TypeScript type from the Zod schema — never maintain both separately.

```
import { z } from 'zod';
// Single source of truth
const UserSchema = z.object({
```

```
  id: z.number(),
  name: z.string(),
  email: z.string().email(),
});
// Type is automatically kept in sync
type User = z.infer<typeof UserSchema>;
```

#46 — Type errors only in production build, not in editor

Solution

Run tsc --noEmit as part of CI. Editor uses tsserver which may use a different tsconfig.

```
// package.json
{ "scripts": {
    "typecheck": "tsc --noEmit",
    "build": "tsc",
    "ci": "npm run typecheck && npm run build"
  }
}
```

#47 — esModuleInterop and allowSyntheticDefaultImports

Solution

Enable both in tsconfig to import CommonJS modules with default import syntax.

```
{ "compilerOptions": {
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true
  }
}
// Without: import * as path from 'path';
// With:    import path from 'path'; // works now
```

#48 — satisfies operator vs type annotation

Solution

Use 'satisfies' when you want type checking but also want TypeScript to infer the narrowest type.

```
// Annotation: loses narrow type
const colors: Record<string, string | number[]> = {
  red: [255,0,0], blue: '#0000ff'
};
colors.red; // string | number[] — too wide

// satisfies: keeps narrow type
const colors2 = {
  red: [255,0,0], blue: '#0000ff'
} satisfies Record<string, string | number[]>;
colors2.red;  // number[] — correct!
colors2.blue; // string  — correct!
```

#49 — Template literal types for API route typing

Solution

Use template literal types to enforce URL shape at compile time.

```
type UserId = number;
type ApiRoute = `/api/users/${UserId}` | '/api/users' | '/api/health';

function call(route: ApiRoute) { return fetch(route); }
call('/api/users');    // OK
call('/api/health');   // OK
// call('/api/posts'); // Error — not in union
```

#50 — Typing event emitters

Solution

Use a generic typed event emitter interface to keep event names and payloads in sync.

```
type Events = {
  'user:created': User;
  'user:deleted': { id: number };
  'error': Error;
};
class TypedEmitter<T extends Record<string, any>> {
  on<K extends keyof T>(event: K, handler: (data: T[K]) => void): this { return this; }
  emit<K extends keyof T>(event: K, data: T[K]): void {}
}
const emitter = new TypedEmitter<Events>();
emitter.on('user:created', (user) => console.log(user.name)); // fully typed
```

#51 — Typing Object.entries and Object.fromEntries

Solution

Object.entries loses type info — cast the result or use typed helper functions.

```
const config = { host: 'localhost', port: 3000 };
// Object.entries returns [string, any][] — loses types
// Typed helper:
function typedEntries<T extends object>(obj: T): [keyof T, T[keyof T]][] {
  return Object.entries(obj) as any;
}
const entries = typedEntries(config);
// entries[0] is ['host' | 'port', string | number]
```

#52 — Typing environment variables with a schema

Solution

Parse and validate process.env at app startup to get typed, guaranteed env vars.

```
import { z } from 'zod';
const Env = z.object({
  NODE_ENV: z.enum(['development','test','production']),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
});
export const env = Env.parse(process.env);
// env.PORT is number, env.NODE_ENV is 'development'|'test'|'production'
```

#53 — Deeply nested optional chains causing type widening

Solution

Extract intermediate values and narrow each level to avoid overly wide union types.

```
// Deeply nested optional — type becomes very wide
const city = user?.address?.city?.name;
// Extract and narrow:
const address = user?.address;
if (!address) return;
const city2 = address.city?.name ?? 'Unknown'; // string
```

#54 — Type narrowing lost after async boundary

Solution

Narrowing is not preserved across await points. Re-narrow after each await.

```
async function process(val: string | null) {
  if (!val) return;
  // val is string here
  await doSomething();
  // val is still string — TS knows, but consider re-checking
  // if external mutation is possible, re-narrow:
  const current = val; // capture before await if needed
}
```

QUICK REFERENCE CHEAT SHEET

Most Used Utility Types at a Glance

Type	Syntax	Example Result
Partial	Partial<User>	All props optional
Required	Required<Partial<User>>	All props required
Readonly	Readonly<User>	All props readonly
Pick	Pick<User,'id'|'name'>	{ id, name }
Omit	Omit<User,'id'>	User without id
Record	Record<string,User>	{ [key: string]: User }
NonNullable	NonNullable<T|null>	T
ReturnType	ReturnType<typeof fn>	fn's return type
Parameters	Parameters<typeof fn>	fn's param types as tuple
Awaited	Awaited<Promise<User>>	User

tsconfig Strict Flags

Flag	What It Catches
strict	Enables all strict checks below
strictNullChecks	Catches potential null/undefined access
strictFunctionTypes	Enforces covariance/contravariance
strictBindCallApply	Types bind/call/apply correctly
noImplicitAny	Disallows implicit any
noImplicitThis	Disallows implicit any for 'this'
noUncheckedIndexedAccess	Index access returns T | undefined
exactOptionalPropertyTypes	Distinguishes missing from undefined

This guide covers what the market actually uses day-to-day based on the 80/20 principle. For more depth, visit: typescriptlang.org/docs